

Prueba - Reconocimiento de imágenes con redes neuronales convolutivas

Clasificación de dígitos manuscritos con CNN en Keras

Caso aplicado: automatización de formularios escritos

1. Explicación conceptual: estructura y utilidad de una CNN

Una red neuronal convolutiva (Convolutional Neural Network, CNN) es una arquitectura especialmente adecuada para datos con estructura espacial, como imágenes. A diferencia de una red totalmente conectada que trataría cada píxel de forma independiente desde el inicio, una CNN aplica filtros pequeños sobre regiones locales de la imagen. Estos filtros comparten parámetros a lo largo de toda la superficie y permiten aprender patrones como bordes, trazos, esquinas y formas.

Capa	Función	Utilidad en reconocimiento de dígitos
Conv2D	Aplica filtros convolucionales y genera mapas de características.	Detecta trazos y patrones locales del dígito.
MaxPooling2D	Reduce las dimensiones espaciales conservando activaciones relevantes.	Disminuye costo y aporta cierta robustez a pequeñas variaciones.
Flatten	Convierte los mapas 2D en un vector.	Prepara las características para la clasificación final.
Dense	Combina características mediante conexiones completas.	Aprende una frontera de decisión entre los dígitos.
Dropout	Desactiva neuronas aleatoriamente durante entrenamiento.	Regulariza y reduce sobreajuste.
Softmax	Convierte logits en probabilidades para 10 clases.	Entrega la probabilidad estimada de los dígitos 0-9.

Aplicaciones prácticas: OCR y lectura automática de formularios, clasificación de documentos, inspección visual industrial, reconocimiento de objetos y apoyo al análisis de imágenes médicas.

Esquema del modelo optimizado

Esquema de la CNN optimizada

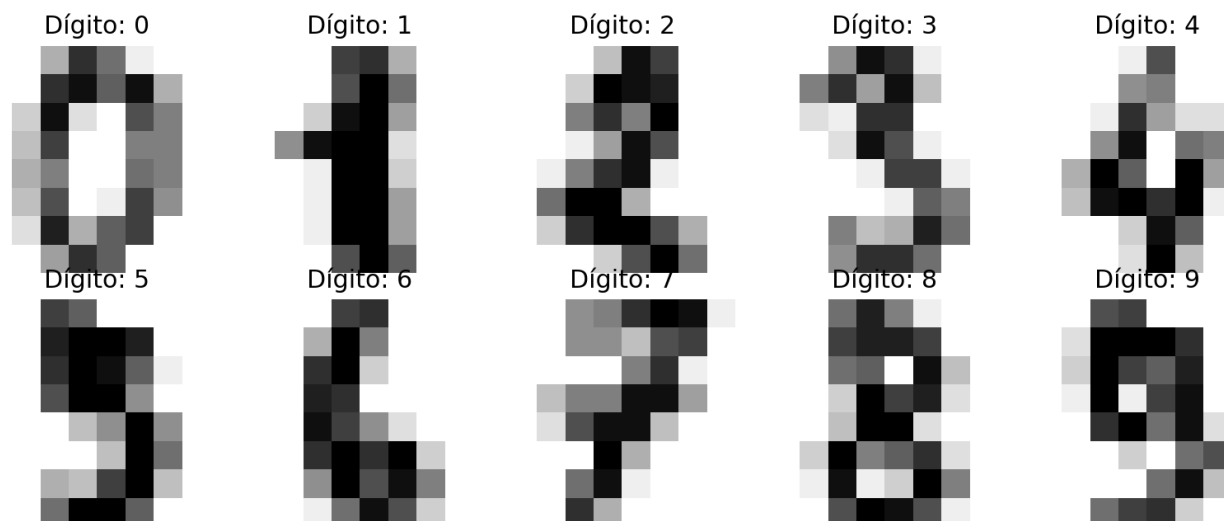


2. Dataset y preprocesamiento

El archivo proporcionado contiene 1797 observaciones, 64 valores de píxel y una columna label. Los 64 valores se reorganizan como imágenes de 8x8 píxeles con un canal: (n, 8, 8, 1), tal como solicita la prueba. Los píxeles están codificados entre 0 y 16, por lo que se normalizan dividiendo por 16 para obtener valores entre 0 y 1.

La partición estratificada utilizada dejó 1149 observaciones para entrenamiento, 288 para validación y 360 para prueba. El conjunto de prueba se mantuvo separado hasta la evaluación final.

Ejemplos del dataset: imágenes 8x8 en escala de grises



3. Implementación del modelo CNN

Se implementaron dos modelos con Keras para disponer de una comparación antes/después: una CNN baseline y una CNN optimizada.

Modelo	Arquitectura resumida	Objetivo
Baseline	Conv2D(16) -> MaxPooling -> Flatten -> Dense(32) -> Softmax	Referencia inicial con arquitectura simple.
Optimizada	Conv2D(32) -> MaxPooling -> Conv2D(64) -> MaxPooling -> Flatten -> Dense(64) -> Dropout(0,30) -> Softmax	Mayor capacidad y regularización.

Configuración de entrenamiento: loss sparse_categorical_crossentropy, métrica accuracy y optimizador Adam. La salida tiene 10 neuronas con activación softmax, una por cada dígito.

```
model = keras.Sequential([
    layers.Input((8, 8, 1)),
    layers.Conv2D(32, 3, activation='relu', padding='same'),
    layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, activation='relu', padding='same'),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.30),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer=keras.optimizers.Adam(0.001),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

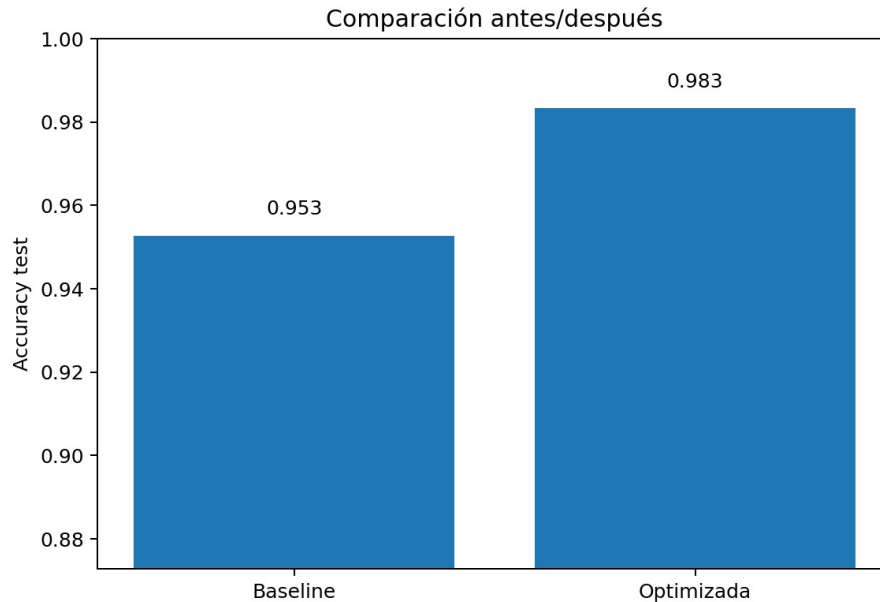
4. Evaluación y optimización

Se aplicaron y analizaron más de dos técnicas de mejora, cumpliendo el criterio de optimización de la rúbrica:

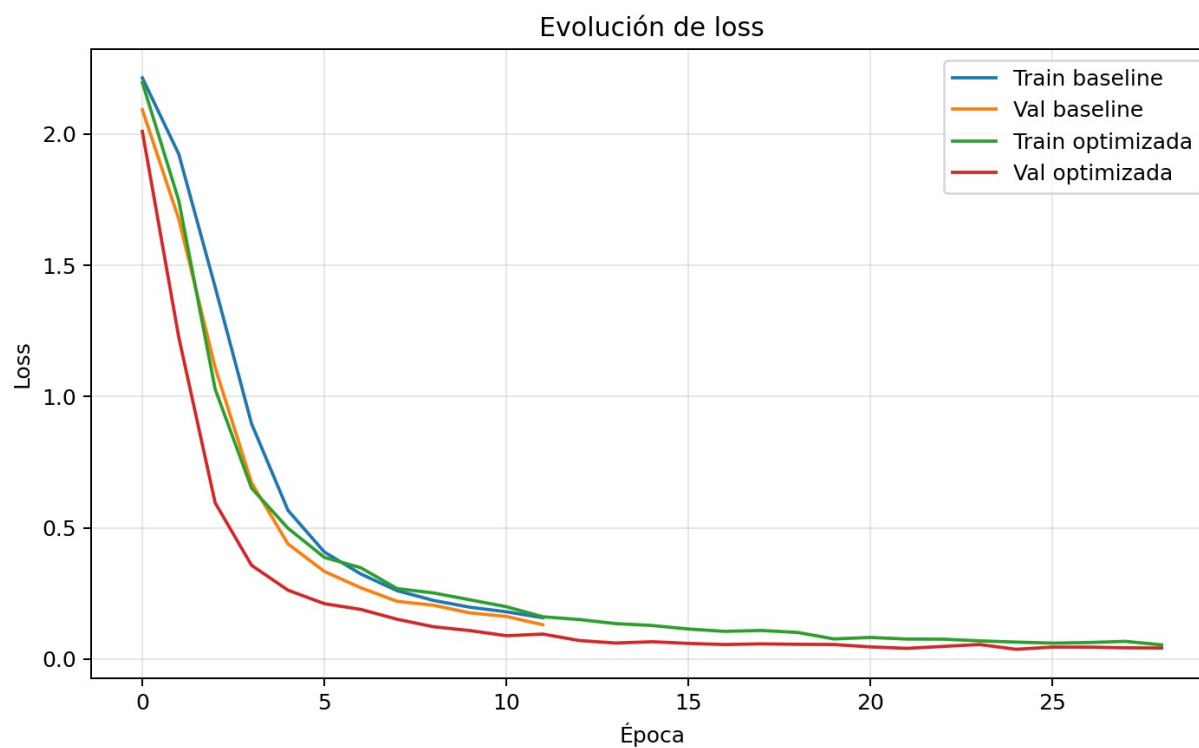
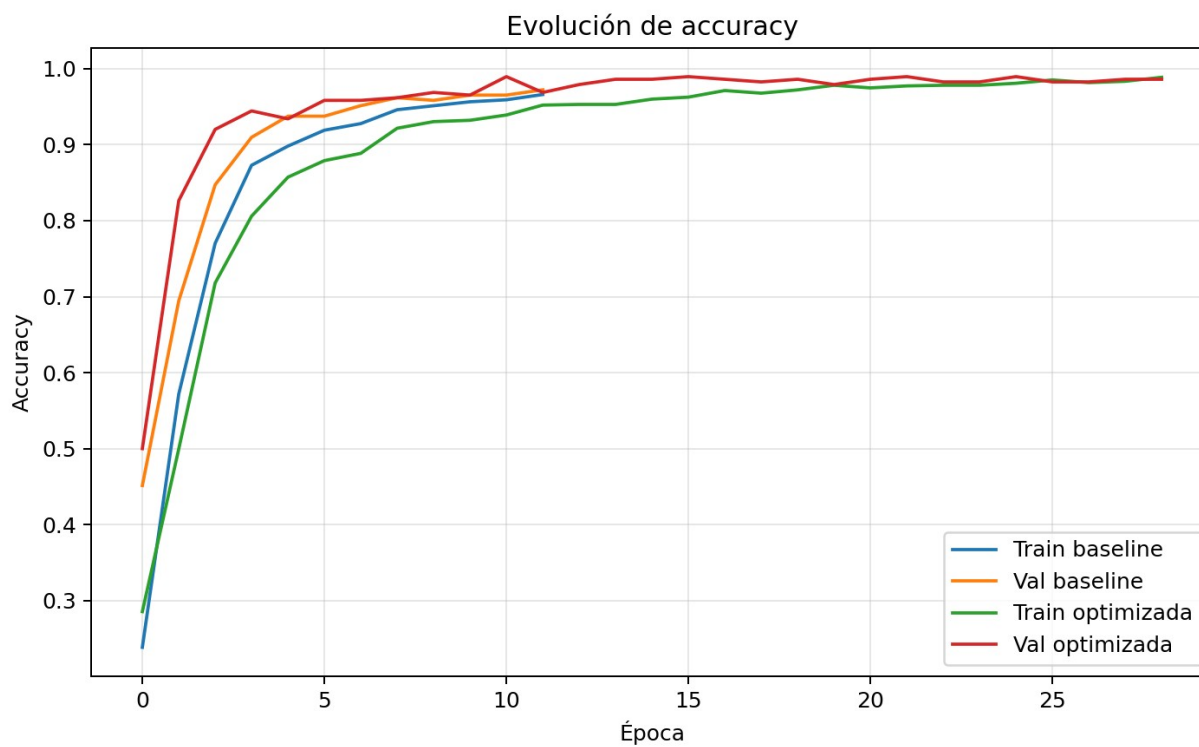
- Aumento de capacidad: dos bloques convolucionales con 32 y 64 filtros.
- Dropout de 30% como regularización para disminuir dependencia excesiva de neuronas específicas.
- EarlyStopping monitoreando validation loss, con restauración de los mejores pesos.
- ReduceLROnPlateau para disminuir el learning rate cuando la validación se estanca.
- Mayor máximo de épocas para permitir convergencia, manteniendo EarlyStopping como control.

Modelo	Accuracy test	Loss test	Épocas	Cambio accuracy
Baseline	95.28%	0.1615	12	Referencia
Optimizada	98.33%	0.0572	29	+3.06 pp

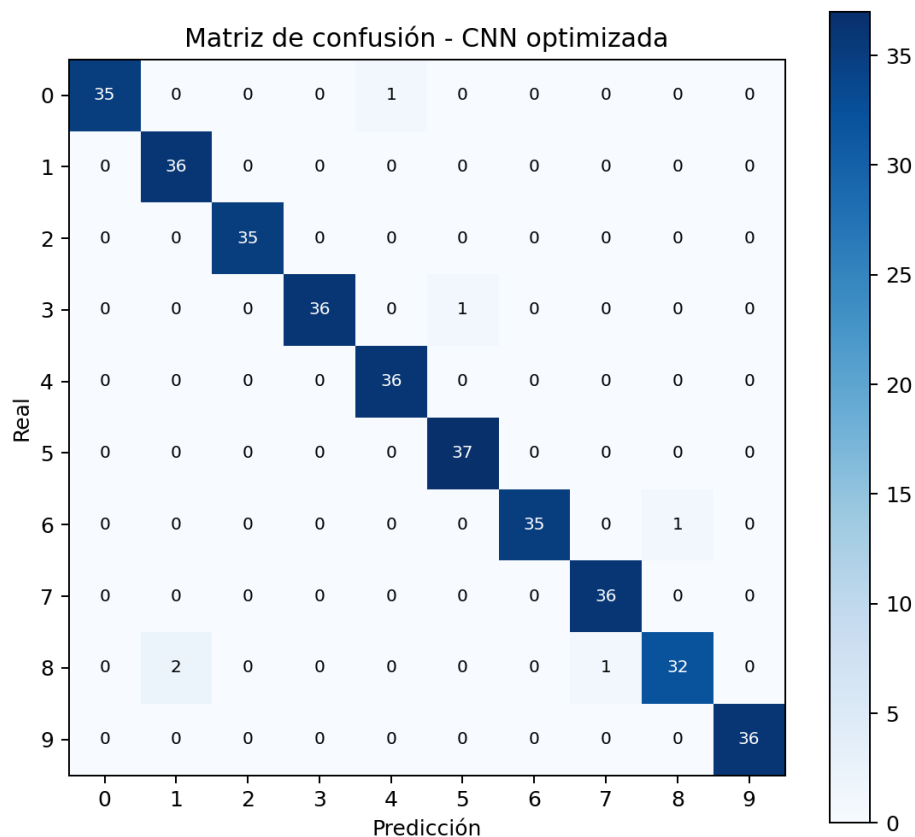
Resultado principal: la accuracy aumentó desde 95.28% hasta 98.33%, una mejora de 3.06 puntos porcentuales. El loss también disminuyó desde 0.1615 hasta 0.0572. Esto evidencia una mejora real sobre el conjunto de prueba utilizado.



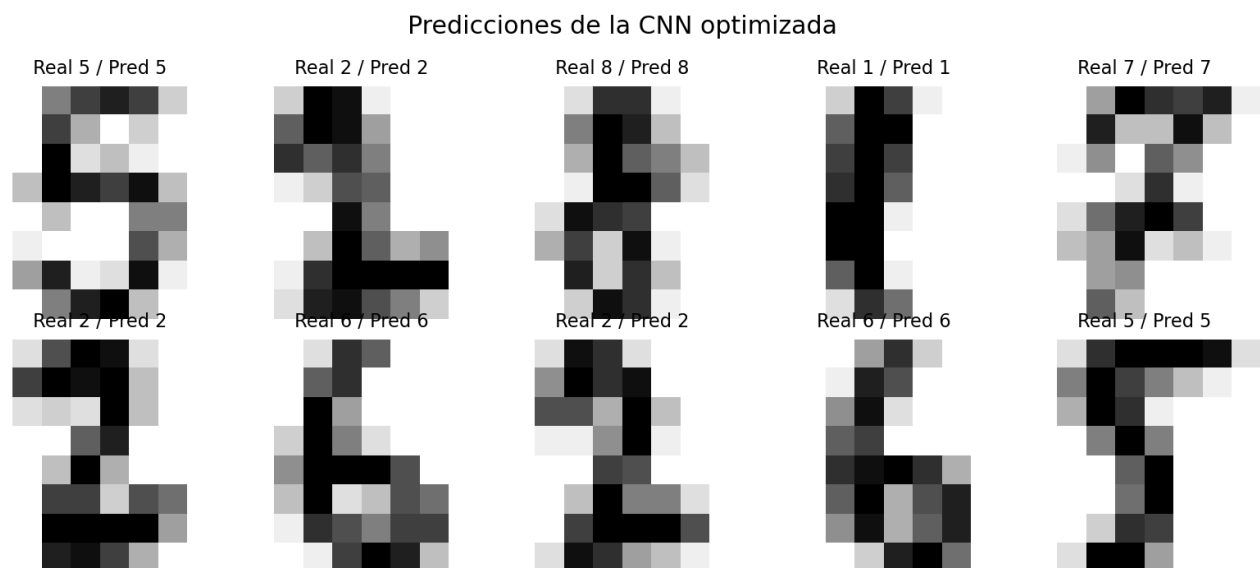
5. Gráficos de entrenamiento



6. Matriz de confusión e interpretación



La matriz de confusión permite identificar qué clases se clasifican correctamente y qué pares de dígitos generan errores. En un sistema real de automatización de formularios, este análisis es más informativo que observar únicamente la accuracy global, porque permite detectar dígitos que requieren más datos o un tratamiento específico.



7. Reflexión crítica sobre las técnicas de mejora

La mejora no proviene de una única modificación, sino del equilibrio entre capacidad y regularización. El segundo bloque convolucional permite representar patrones más complejos, mientras Dropout reduce el riesgo de

sobreajuste en la etapa densa. EarlyStopping limita entrenamiento innecesario y ReduceLROnPlateau facilita ajustes más finos cuando el aprendizaje se estabiliza.

El dataset es relativamente pequeño y las imágenes son de solo 8x8 píxeles, por lo que una CNN demasiado profunda no sería conveniente. Para un sistema productivo se recomienda ampliar el corpus, incorporar data augmentation, validar con imágenes capturadas en condiciones reales y repetir los experimentos con varias semillas.

8. Conclusión

La CNN optimizada alcanzó 98.33% de accuracy en el conjunto de prueba, superior al 95.28% de la CNN baseline. El ejercicio demuestra el flujo completo requerido: reshape a (n, 8, 8, 1), normalización, definición de Conv2D/Pooling/Flatten/Dense, entrenamiento, análisis de accuracy y loss, matriz de confusión, optimización y reflexión. La solución es técnicamente apropiada como prototipo para reconocimiento de dígitos manuscritos en una futura automatización de formularios.

Nota de ejecución: La ejecución de referencia se realizó con Keras 3.13.2 usando backend torch. El notebook también incluye compatibilidad con TensorFlow/Keras en Google Colab.